MedSim VRAI — Educator Management GUI

A best-in-class interface to set up, operate, test, and recover the training system — with the current control room as a default fallback. Report · design critique · 3 examples + consolidated best · resumability · coding plan.

Date: 2026-06-13 · **Inputs:** the attached layered-ecosystem direction (*SimMedVRAI GUI-1.pdf*) · `Memory_management.MD` §3 (module boundaries), §7 ADR-0018 (pause/resume) · the shipped two-stage control room (FR-005) · `preflight.sh` · FR-010 (readiness ping) · **Companions:** MedSimVRAI_GUI-Redesign.xlsx (comparison + screen/state inventories + backlog) · PLAN-2026-06-13-gui-mission-control.md (Claude Code coding plan)
Two hard requirements honored throughout: (1) the GUI is an **alternative** — the classic control room stays the **default fallback**, always one click away; (2) **no data is lost** on pause / stop / restart (Memory_management ADR-0018 extended from the tablet to the portal).

1 · The educator's problem (why a GUI)

The system is now feature-rich — avatars, audio stations, medication ordering, staged errors, shift handoff, multi-patient rooms, devices, debrief. That power is exposed through one dense control page plus shell scripts. For a non-technical educator under time pressure before a class, three frictions dominate:

- **Startup time & uncertainty.** Preflight is a terminal script; "is everything working?" is answered by checking several places. Pairing a tablet, trusting a certificate, warming the speech model — each is a separate, manual, easy-to-miss step.
- **Restart fragility.** Any portal restart **wipes the in-memory session** — the educator must reconfigure from scratch mid-prep (observed repeatedly during development). There is no "resume where I left off."
- **Configuration is expert-shaped.** The control page assumes you know the model (which persona is the patient, avatar vs audio, which characters belong to a scenario). There is little guidance, few presets, and easy ways to misconfigure.

A purpose-built educator GUI should turn each of these into a guided, status-first, recoverable flow — and measurably cut time-to-training.

2 · Critique of the attached direction

The attached diagram is a **layered ecosystem model**: Simulation control → shared *characters* (doctor, charge nurse, supervisor, allied health) → shared *resources* (nursing station, charting, med cart) → *rooms* → *individual patients* (avatar/manikin, family, devices). As a conceptual taxonomy it is genuinely strong and should be preserved.

What it gets right

- **Correct mental model.** It separates *shared* staff/resources (used across patients) from *individual* patients — exactly the distinction multi-patient/charge-nurse training needs (and that FR-007 is filed against).

Where it must be improved (best-in-class gaps)

- **It is a taxonomy, not an interaction.** It shows *what exists*, not *how an educator goes from blank → running*. There is no setup **flow**, no **guidance**, no defaults.

- **Composable hierarchy.** Ecosystem → room → patient → device mirrors how a unit is actually organized; it scales from one patient to a ward.
- **Device + family inclusion.** Treating call bell / IV / telemetry / vent and family/support as first-class is right.

- **No status / readiness layer.** Nothing tells the educator whether each tablet/character/system is *working* — the single biggest pre-class anxiety.
- **No recovery model.** No "resume," no restart-safe state — the exact friction the system suffers today.
- **No progressive disclosure.** A flat board shows everything at once; novices need a guided path, experts need the full board. Best practice serves both.
- **No error/empty states or testing.** "Test the system before training" is unaddressed.

The redesign therefore **keeps the layered model as the structural backbone** and layers on the three things best-in-class operations consoles always add: **guided setup**, **status-first operation**, and **recoverability**.

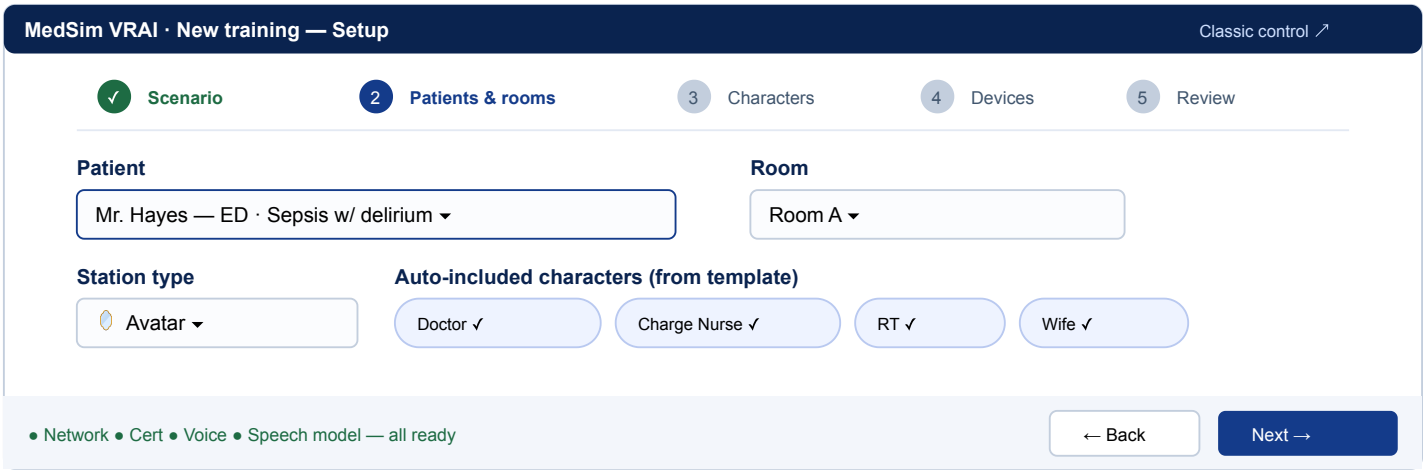
3 · Best-in-class methodologies applied

Principle (source pattern)	What it means here
Guided setup / wizard (NN/g "wizard" pattern; appliance setup)	A linear, validated, pull-down-driven path from a scenario template to a running session — minimizes decisions and misconfiguration; "you can't reach Launch until you're ready."
Single pane of glass / status dashboard (NOC & ops consoles)	One screen shows every device, character and subsystem as a traffic-light tile with health + a one-click action. Answers "is it working?" instantly.
Progressive disclosure (NN/g)	Novice → the wizard (few choices); expert → the ecosystem board (full control). Same data, two depths.
Templates & sane defaults (config-as-presets)	Scenario templates pre-fill patient, characters, devices, modules (already in `sample_scenarios.json` / activities). The educator confirms, doesn't author.
Recoverability / autosave (Memory_management ADR-0018; "never lose work")	Structured session state is snapshotted continuously and restored on boot — pause/stop/restart resume exactly where you were.
Guardrails & validation (poka-yoke)	Readiness gates the Launch; destructive actions confirm; the cert/network state is shown, not assumed.
One-click remediation (self-service ops)	Each red tile offers its fix inline — Warm, Re-pair, Restart, Open instructions — instead of a terminal.
Graceful fallback (no dead ends)	"Switch to classic control room" is always present; the new GUI never traps the educator.

4 · Three GUI examples (worked references)

Three distinct approaches, each strongest for a different need. Section 5 consolidates them.

Example A — Guided Launch Wizard fastest setup

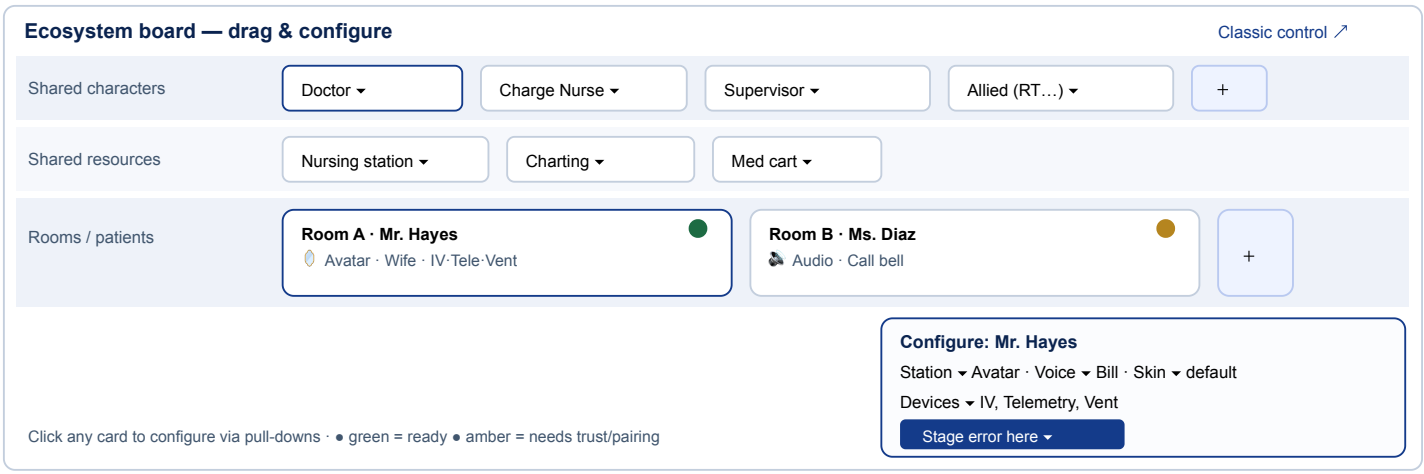


Wizard: a stepper + pull-downs + template auto-fill + a readiness strip that gates the final Launch. Best for novices and speed.

Strengths. Lowest cognitive load; near-impossible to misconfigure; fastest blank→running; perfect for the "reduce startup time" goal; readiness baked into the flow.

Limits. Linear; weaker for ad-hoc mid-session changes and for visualizing a multi-patient unit; an expert may find it slow for the 10th run.

Example B — Ecosystem Board structure & multi-patient

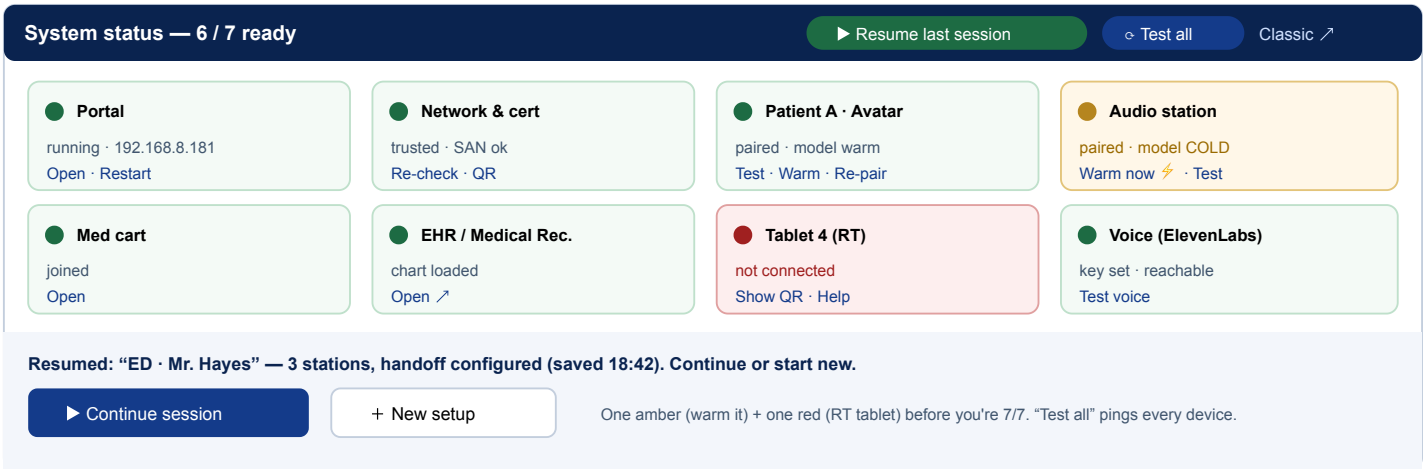


Board: the attached layered model made interactive — click a card, configure via pull-downs, see per-entity readiness dots. Best for multi-patient units and spatial thinkers.

Strengths. Matches the educator's mental model directly; superb for multi-patient/charge-nurse and ad-hoc changes; everything visible and directly manipulable; honors the attached direction.

Limits. More clicks and screen real estate; steeper for novices; without a guided path a beginner can stall on "where do I start?".

Example C — Readiness Cockpit operate · test · restart



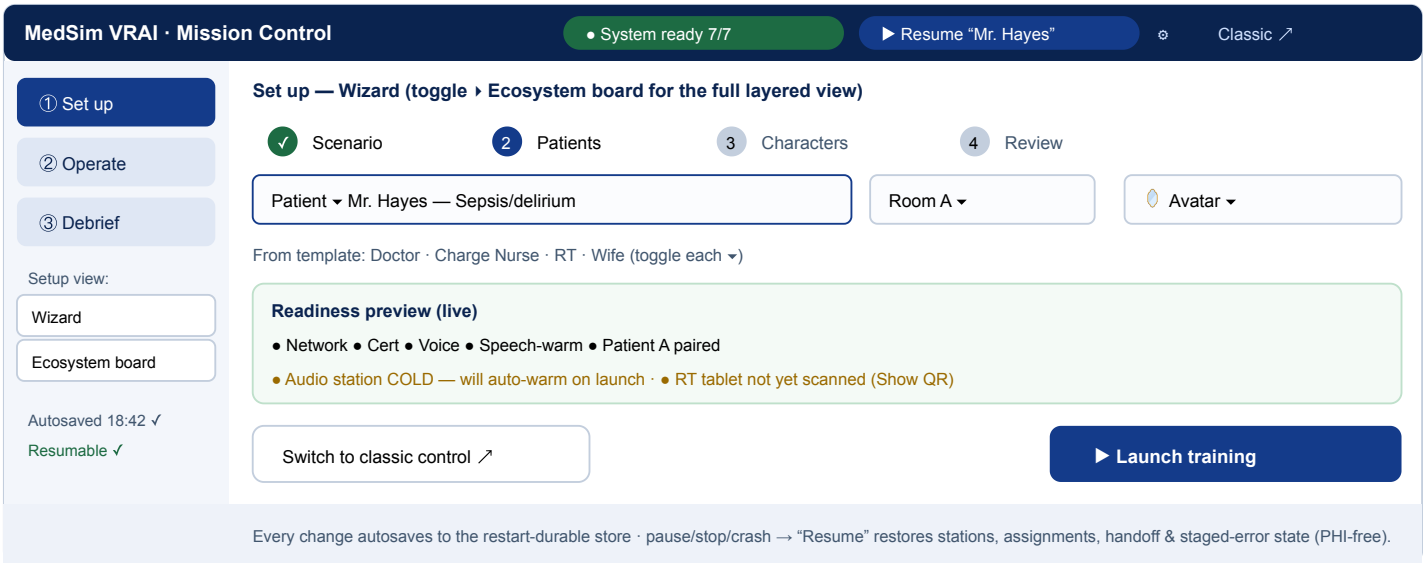
Cockpit: every device/character/subsystem as a traffic-light tile with inline fixes, a "Test all," and a "Resume last session" banner. Best for operating, testing, and recovering — directly targets startup time + restart.

Strengths. Answers "is it working?" in one glance; one-click Warm/Test/Re-pair/Restart replaces the terminal; the Resume banner kills restart friction; ideal pre-class and mid-session.

Limits. It operates an *already-configured* session — it is not itself a setup authoring tool (needs the wizard/board for that).

5 · Consolidated best example — "Mission Control"

No single example is sufficient: A sets up fast, B models structure, C operates & recovers. The consolidated design is **one app, three modes** over a shared session model, with a persistent readiness bar, the classic control room as a one-click fallback, and a portal-side resumability layer underneath everything.



Mission Control: a 3-mode shell (Set up · Operate · Debrief) over one resumable session model. Setup offers Wizard *or* Ecosystem board; a live readiness preview gates Launch; Operate is the Cockpit; the classic control room is always one click away; everything autosaves and resumes.

Why this is the best synthesis

- **Serves novice and expert** (progressive disclosure): the Wizard for speed, the Ecosystem board for full structural control — same session model underneath.
- **Status-first**: the readiness bar + Cockpit answer "is it working?" everywhere, and gate Launch so a class never starts half-broken.
- **Recoverable**: the resumability layer makes restart a non-event — the single biggest time sink today becomes a one-click "Resume."
- **Safe by construction**: pull-downs + templates + validation prevent misconfiguration; the classic control room is the guaranteed fallback (requirement honored).
- **Honors the attached direction**: the Ecosystem board *is* the layered model, now interactive.

6 · Resumability — no data lost on pause / stop / restart

This is both a hard requirement and the highest-leverage fix. `Memory_management.MD §7 ADR-0018` already defines the pattern on the **tablet** (each module implements `Resumable<T>` ; `memory_state` aggregates `SnapshotHooks` → one structured-clone-safe `SessionState` blob to IndexedDB; `resumeAll()` on boot; **PHI free-text is never persisted**). The gap is the **portal**: the `ControlRoom` / `ControlSession` + handoff + staged-error + med-board + assignments live in memory and are wiped on restart.

The fix (one new ADR, extends 0018 server-side): mirror the same contract on the portal. Each stateful module (`control_session`, `med_orders`, `med_errors`, `handoff`, voice/skin assignments) exposes a **PHI-free** structured snapshot; a `session_state` aggregator writes ONE blob to the **already restart-durable** SQLite (`~/.medsim/v7/medsim.db` — the EHR chart already survives restarts there) on every mutation + on shutdown; on boot the portal **auto-restores** the last session. The GUI's "Resume" simply surfaces it. Trainee free-text stays out of the snapshot (ADR-0014 fail-closed).

Result: a Python change / crash / power blip no longer costs the educator their setup — they reopen Mission Control and press **Resume**. This also subsumes FR-010's intent (post-restart readiness) and removes the "fresh Setup after every restart" friction seen throughout development.

7 · Cutting startup time (the explicit goal)

- **Templates & presets** — one pull-down (scenario) pre-fills patient + characters + devices + modules; the educator confirms instead of authoring.
- **Parallel readiness + one-click Warm** — the cockpit pings all devices at once and warms the speech model on demand, so the first take is never cold.
- **Resume** — the biggest single saver: skip setup entirely after a restart.
- **Preflight-as-a-service** — the terminal checks become a "Test all" button with inline fixes (no shell, no context-switch).
- **Guided pairing** — a tablet that isn't connected shows its QR + plain-language fix in its own red tile, instead of silent failure.

8 · Risks, fallback & accessibility

- **Fallback is mandatory and built-in** — "Switch to classic control room" on every screen; the new GUI is additive, the control room remains the source of truth and the default.
- **Don't fork the backend** — the GUI is a new front-end over the SAME portal APIs (control/meds/errors/handoff/devices) the control room uses; no parallel logic to drift.
- **Snapshot safety** — strictly structured, PHI-free state; never persist trainee free-text; version the blob and tolerate schema drift on resume.
- **Accessibility** — keyboard-navigable, high-contrast traffic lights paired with text/shape (not color alone), large touch targets for a cart-side tablet.
- **Scope discipline** — module-per-PR and an ADR for the server-snapshot contract, per `Memory_management §0/§3/§7`.

9 · Recommendation

Build **Mission Control** in the staged plan (companion coding plan), **foundation-first**: ship the **portal resumability layer** and the **readiness/health API** before any new pixels — those alone remove the worst friction and are reusable by the classic control room too. Then layer the GUI shell, the Cockpit (Operate), the Wizard (Set up), and the Ecosystem board, keeping the classic control room as the default fallback throughout. The supporting Excel

carries the example comparison, the screen/state inventories, and the build backlog; the coding plan specifies it to the file.